PROJECT OVERVIEW & STUDY GUIDE

TensorSwarm

A peer-to-peer GPU cluster that turns the ordinary NVIDIA PCs in a college lab into one private, Colab-style compute service, installed from a single USB stick.

1 USB
installs every PC, no internet or admin needed for the software

~3 min
to install a PC from the stick

Any PC
opens the dashboard in a browser and sends GPU work

0 GPUs
used by the Master: it only orchestrates

Contents

1. The problem and the idea
2. Architecture at a glance
3. Technology stack: what and why
4. How it works, step by step
5. Running code on a remote GPU
6. The data layer (datasets)
7. Key engineering decisions
8. Features
9. Measured results
10. Use cases
11. Advantages
12. Drawbacks and limitations
13. Comparison with alternatives
14. Likely questions, with answers
15. Glossary

Stack: Windows 10/11 · Tailscale (WireGuard) · Ray 2.35 · FastAPI · PyTorch 2.5.1 (CUDA 12.1) · portable CPython 3.10.15 · Next.js 14 · CodeMirror 6 · NVML

1. The problem and the idea

College labs are full of PCs with decent NVIDIA GPUs that sit idle, while students training models queue for Google Colab's free tier, fight its time limits, or run out of VRAM on their laptops.

What stands in the way of using those lab GPUs

- **Locked-down PCs:** no admin rights, no Docker, no WSL, and installs are wiped or forbidden.
- **Messy networks:** PCs sit behind campus NAT and firewalls; laptops join from hotspots. Machines can't simply address each other.
- **Setup cost:** installing Python, CUDA-enabled PyTorch and a cluster framework by hand on 20 PCs takes hours, and versions drift.
- **No shared interface:** even with everything installed, there is no single place to write code and send it to "a free GPU".

The idea

> **TensorSwarm in one sentence**
>
> Carry a complete, portable Python + PyTorch + Ray environment on one USB stick, install it on each lab PC in minutes, connect the PCs with an encrypted mesh VPN (Tailscale), and give everyone a browser dashboard with a Colab-like notebook that runs code on whichever PC has a free GPU.

One PC is the **Master** (the coordinator). Every PC with a GPU runs a **Worker** that offers its GPU to the pool. Anyone on the network opens `http://<MASTER_IP>:8000`, writes Python, and it runs on a lab GPU, not on their own machine.

Goals the design was held to

Goal	How it is met
Zero-install on lab PCs	Portable CPython copied to `C:\TensorSwarm`; no system changes except optional firewall rules.
Works across networks	Tailscale gives each PC a stable `100.x.y.z` address and punches through NAT.
Fast to roll out with one USB	The 31,669-file environment travels as one 2.5 GB archive; each PC unpacks it in ~3 min.
Familiar to students	A notebook modelled on Google Colab: cells, persistent variables, `!pip`, `!nvidia-smi`.
Robust on bad Wi-Fi	Every transfer resumes where it stopped and is verified with SHA-256.

2. Architecture at a glance

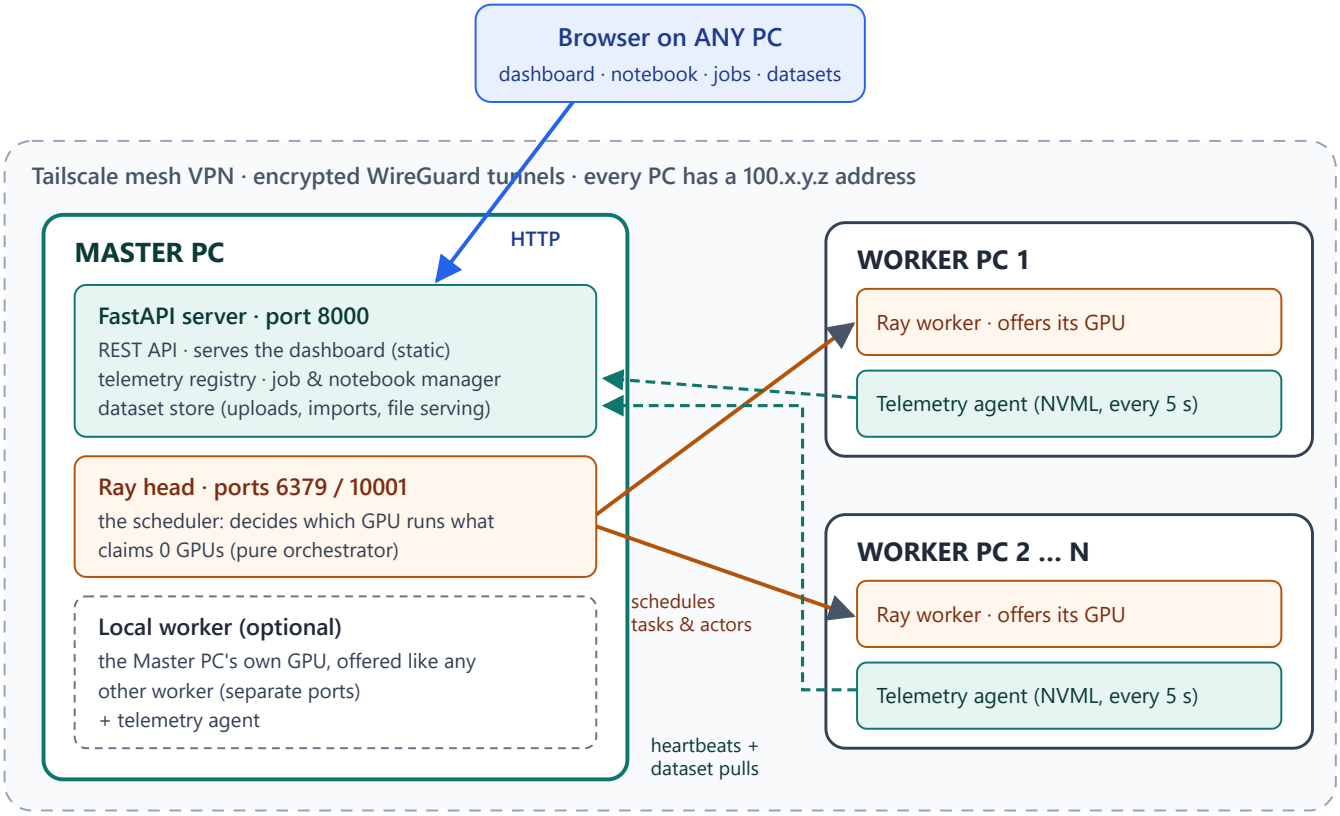


Figure 1: Components and the traffic between them. All traffic stays inside the Tailscale network.

Component	Runs on	Role
FastAPI server (`server.py`)	Master	The "front desk": receives code and telemetry, talks to Ray, serves the web UI and datasets, all on port 8000.
Ray head	Master	The cluster's brain. Knows every node's free GPU share and places work. Started with `--num-gpus=0` so it never takes a GPU itself.
Ray worker	Every GPU PC	Joins the head over Tailscale and runs the Python processes that execute user code on the local GPU.
Telemetry agent (`agent.py`)	Every PC	Reads the GPU through NVIDIA's NVML library (load, VRAM, temperature, power, clocks, throttling) and posts a heartbeat every 5 s.
Dashboard	Any browser	Next.js app compiled to static files, served by FastAPI. No Node.js needed anywhere at runtime.

3. Technology stack: what and why

Technology	Used for	Why this choice
Tailscale (WireGuard)	Private network between all PCs	Zero-config mesh VPN: NAT traversal, encryption and stable IPs without touching campus routers. Falls back to relays (DERP) when a direct path is impossible.
Ray 2.35	Distributed scheduling and execution	Python-native: a function becomes a remote task with `@ray.remote(num_gpus=1)`. Tracks fractional GPU shares, node affinity, actors (stateful processes). Supports Windows nodes.
FastAPI + Uvicorn	REST API, file transfer, serving the UI	Async Python web framework with automatic API docs at `/docs`; lives in the same process as the Ray client.
PyTorch 2.5.1 (CUDA 12.1)	The GPU workload	Most common deep-learning framework in courses; bundles its own CUDA runtime, so lab PCs only need an NVIDIA driver (≥ 527.41).
Portable CPython 3.10.15 (python-build-standalone)	The runtime on every PC	A relocatable Python that runs from any folder without installation. A virtualenv was rejected: it hard-codes paths and drive letters.
NVML (nvidia-ml-py)	GPU telemetry	The same library `nvidia-smi` uses; exact per-GPU metrics without parsing text.
Next.js 14 + Tailwind CSS	Dashboard	React framework; exported as plain static files so the Python server can serve it.
CodeMirror 6	Notebook code editor	Python syntax highlighting and keyboard shortcuts; bundled, so it works offline.
PowerShell + `.cmd` wrappers	Install, start, stop scripts	Built into every Windows PC; `.cmd` wrappers bypass execution-policy blocks.
tar, **robocopy** (built into Windows)	Packing and copying the environment	One sequential archive copies far faster than 31,000 small files; robocopy copies only changes.

> **Version pinning matters**
>
> Ray only connects nodes running the *exact same* Python and Ray versions. Because every PC is installed from the same archive, all nodes are byte-identical. One real bug found during development: `click` ≥ 8.3 breaks Ray 2.35's command line, so it is pinned below 8.3.

4. How it works, step by step

4.1 Deployment: one USB stick, many PCs

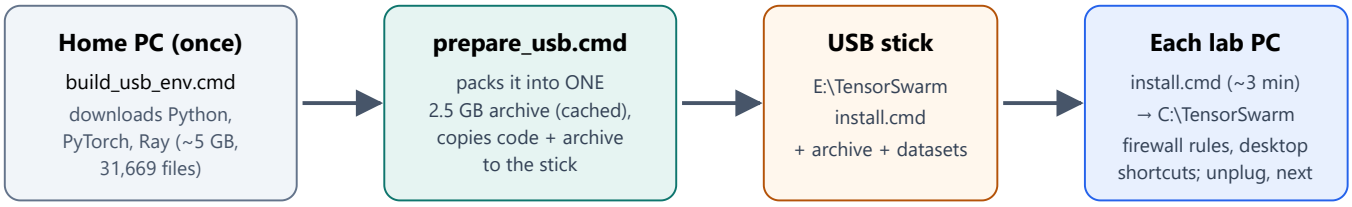


Figure 2: The stick is an installer, not the place the software runs from.

1. **Build (once, at home, ~30–60 min):** `build_usb_env.cmd` downloads a relocatable Python and every package wheel, then installs offline and runs a self-test.
2. **Write the stick:** `prepare_usb.cmd` verifies the environment, packs it into one archive (repacked only when packages change, identified by a hash of `pip freeze`) and copies it plus the code. After code-only changes: `-CodeOnly` takes seconds.
3. **Install each PC:** `install.cmd` unpacks to a staging folder, checks the file count and that `ray` / `torch` import, then swaps it in, so a failed install never breaks a working one. If run as Administrator it adds firewall rules that only allow Tailscale addresses (`100.64.0.0/10`).

4.2 Start-up

1. **Master PC:** the *TensorSwarm Master* shortcut starts the Ray head (0 GPUs), the FastAPI server, and a local worker so the Master PC's own GPU is used too. It prints the Master's Tailscale IP.
2. **Every other PC:** *TensorSwarm Worker* asks for that IP, checks that the head and API are reachable, detects GPUs through NVML, and runs `ray start --address=<MASTER_IP>:6379` plus the telemetry agent.
3. **Anyone:** *TensorSwarm Dashboard* (or `http://<MASTER_IP>:8000`) opens the UI. The Cluster page shows one card per PC with live GPU metrics.

> **A cross-PC bug worth knowing**
>
> Ray makes every worker use the *head's* temp-folder path. By default that is inside the Master user's profile (e.g. `C:\Users\ADMIN\…`), which doesn't exist on other PCs, so workers crashed with "Access is denied". The fix: a user-independent path that exists on every Windows PC, `C:\ProgramData\TensorSwarm\ray` .

5. Running code on a remote GPU

There are two ways to run code. Both go browser → FastAPI → Ray → a worker's GPU.

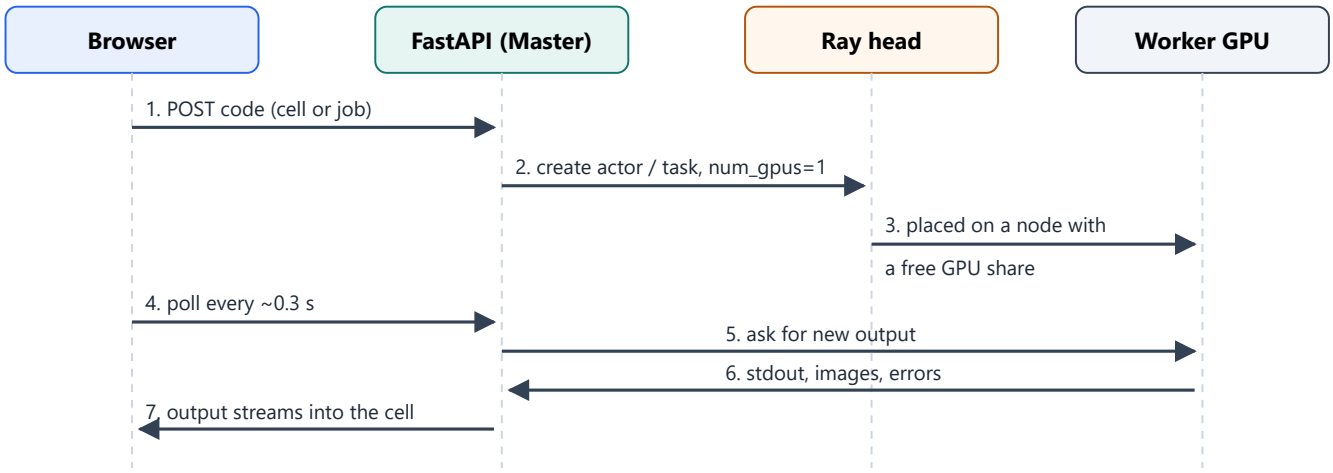


Figure 3: Lifecycle of a notebook cell. Batch jobs follow steps 1–3 and return everything at the end.

Notebook runtime (Colab-style)

- A **Ray actor**: a long-lived Python process pinned to one GPU on one worker.
- Variables persist between cells; **Restart** kills the process.
- Live output, images (`display()` , matplotlib), tracebacks pointing at your cell, peak VRAM per cell.
- `!nvidia-smi` , `!pip install` run on the *worker*.
- **Interrupt** raises `KeyboardInterrupt` inside the running cell.
- Idle runtimes stop after 60 min to free GPUs.

Batch job

- A **Ray task**: runs a whole script once and returns its `result` , stdout and errors.
- Can target one PC, or any free GPU.
- **Run on every GPU** sends the same test to all workers, e.g. to benchmark the whole lab.
- Records the PC, GPU, run time and peak VRAM.

GPU sharing

Ray tracks GPUs as numbers. A runtime can reserve **1** (the whole GPU), **0.5** or **0.25** (shared) or **0** (CPU only). If no share is free, the request fails immediately and names who holds the GPU, instead of waiting silently.

Automatic library install (like Colab)

Before a cell or job runs, its `import` statements are parsed. Any module missing on that worker is installed with pip first (`sklearn` → `scikit-learn` , `cv2` → `opencv-python-headless` , …). Installed packages are **pinned** during the install, so torch, Ray and numpy can never be changed by accident.

> **The hardest bug: CUDA and threads on Windows**
>
> Ray runs actor methods on a thread pool. Touching CUDA from a different thread each time froze the whole process on Windows. The fix was a single dedicated "cell thread" (like Jupyter's main thread) that runs all user code and every CUDA call. Status calls (poll, variables) never touch CUDA, and **Restart** hard-kills the process so a hung GPU is always released.

6. The data layer (datasets)

Code runs on another PC, so training data must be **on that PC**. Campus Wi-Fi is slow and unreliable, and Windows has no NFS, so the data layer uses plain files plus HTTP.

Three options evaluated, and why the answer is all three

Option	Strength	Weakness
A. Upload to the Master, workers pull over HTTP	Works for any data; one source of truth	The Master's uplink is shared by all workers
B. Pre-load on the USB stick	No network at all; the stick visits every PC anyway	Must be planned in advance; FAT32 4 GB file limit
C. Worker downloads from the internet	No load on the Master	Uses shared campus internet; files can change

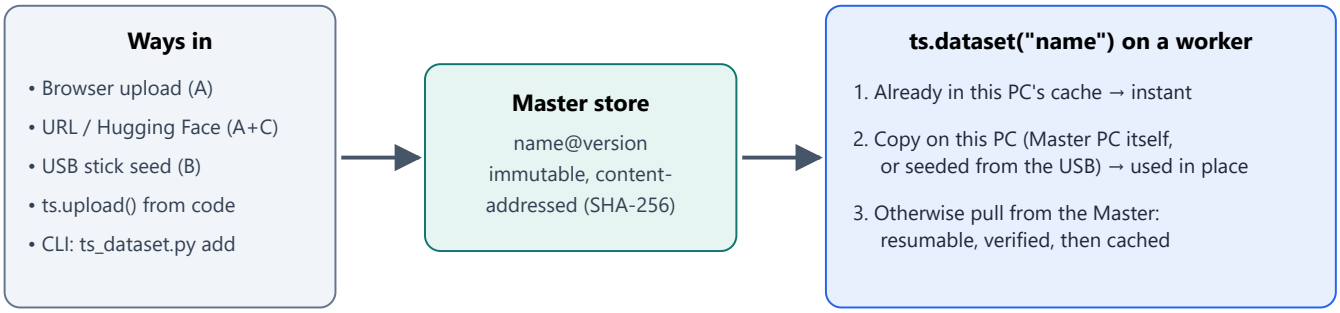


Figure 4: One dataset model, several ways in, and one call to get it on any worker.

Mechanisms (good exam material)

- **Content addressing:** a version id is a hash of every file's path, size and SHA-256. The same content gives the same id, so re-uploads are free and a cached copy can never be stale.
- **Resumable chunked upload:** 8 MB chunks; the Master only appends at the offset it already has, otherwise it replies `409` with the expected offset. Retries and lost replies are harmless; uploads survive a closed tab or a Master restart.
- **Resumable download:** HTTP `Range` requests continue partial files; each file is SHA-256-verified, and corrupted parts are fetched again.
- **Atomicity:** a dataset becomes visible with one folder rename, and a cache entry with one marker file. A crash leaves the old state or the new one, never half.
- **Locking:** an OS file lock means two runtimes on one PC share one download.
- **Data locality:** heartbeats report which datasets each PC holds; jobs that declare datasets prefer such a PC (Ray soft node affinity), but never wait for it.
- **Archives for many small files:** image folders travel as one `.zip` and are extracted on each worker. Per-file overhead dominates on slow links.

7. Key engineering decisions

Decision	Reason / trade-off
USB stick as an **installer**, not a runtime	Copying 31,000 small files to a USB took 10+ minutes; one archive copies at full speed, and code runs from the faster local disk. Cost: ~5 GB of disk on each PC.
Master claims **0 GPUs**	The coordinator never competes for a GPU. Its PC's GPU is still used through a separate local worker.
Portable Python, **no virtualenv**	A venv hard-codes absolute paths; the same folder must work on any PC and drive letter.
Start Ray via `python -m ray.scripts.scripts`	pip's `ray.exe` launcher embeds the drive letter of the build PC.
Dashboard as **static files** served by FastAPI	No Node.js on lab PCs; one port (8000) for UI and API; every PC can use it.
Firewall rules for **Tailscale addresses only**	Other users on the campus LAN can't reach the cluster directly.
Fail fast when no GPU is free	Users see who holds the GPU at once, instead of a 90-second timeout.
Code shipped to workers **by value**	Ray normally pickles functions by module reference; workers don't have `server.py`. By-value pickling (and shipping the data client as source text) means workers need nothing extra.
Plain files, no database	The dataset catalog is rebuilt by scanning folders at start-up; nothing to corrupt or repair.
Auto-install with **pinned constraints**	Convenience of Colab without the risk of pip upgrading torch or numpy and breaking Ray.

8. Features

Cluster page

One card per PC: GPU model, load, VRAM, temperature, power vs. limit, SM/memory clocks, fan, P-state, throttle reasons, PCIe link, CUDA cores, driver/CUDA versions, CPU and RAM, processes on the GPU, and 30 minutes of history charts.

Notebook

Colab-style cells with Shift/Ctrl/Alt+Enter, persistent variables, variable inspector, Markdown cells, images and HTML output, interrupt/restart, `.ipynb` and `.py` import/export, run the notebook as a batch job.

Jobs & Samples

10 built-in test scripts: GPU info, correctness check, usable VRAM, TFLOPS, memory bandwidth, CPU vs GPU, 60 s stress test, CNN training, dataset training, notebook tour. Run on one node or on every GPU.

Datasets

Drag-and-drop upload (pause/resume), URL and Hugging Face import, versions, which PCs hold each dataset, "Copy to all PCs", "Clear worker caches", one-click code snippets.

Operations

Desktop shortcuts (Master, Worker, Dashboard, Stop), one-command USB refresh, self-healing installs, readable logs, a stop script that only touches TensorSwarm's own processes.

Interfaces

REST API with interactive docs at `/docs`; Python client `ts` inside every runtime (`ts.dataset`, `ts.upload`, `ts.download`, `ts.hf_download`).

9. Measured results

Measured during development on the test PC (NVIDIA GTX 1650, 4 GB, Turing, 8 GB RAM), using the built-in samples.

Measurement	Result
Matrix multiply throughput	~2.7 TFLOPS FP32, ~0.42 TFLOPS FP16 (the GTX 1650 has no Tensor Cores)
Usable VRAM	~3.0 GB of 4 GB (the rest is held by Windows and the CUDA context)
CNN training (synthetic 32×32 images, mixed precision)	~2,800 images/s
CPU vs GPU speed-up	~35× (2048×2048 matmul) up to ~380× (sorting 10 M numbers); tiny 64×64 matmul is *slower* on the GPU (launch overhead)
Dataset sample (3-class spirals, MLP)	99.9% test accuracy, ~6 s training on the GPU
Environment size	31,669 files / ~4.8 GB → one 2.48 GB archive
Install a PC from the stick	~3.1 min; reinstall with unchanged environment: seconds
Writing the stick	~13 min the first time; code-only refresh under 1 min

Notebook runtime start	~8–20 s (a Python process importing PyTorch and initialising CUDA)
Automated tests	API suite, data-layer suite (resume, corruption, concurrency, restart) and browser tests in Microsoft Edge all passing

10. Use cases

- **Course labs and assignments:** a class trains models on lab GPUs from their own laptops, with no setup on the laptops beyond Tailscale and a browser.
- **Hyper-parameter sweeps:** run many independent training runs in parallel, one per GPU (different learning rates, seeds, folds of cross-validation).
- **Benchmarking a lab:** "Run on every GPU" compares all PCs side by side and catches broken drivers or failing VRAM.
- **Student projects and hackathons:** a team pools whatever NVIDIA laptops and desktops it has into one shared GPU service for a weekend.
- **Inference and data processing:** batch image classification, embedding generation or feature extraction split across PCs.
- **Teaching distributed systems:** a concrete, observable example of scheduling, heartbeats, fault tolerance, content addressing and resumable transfer.
- **Using a weak laptop:** a laptop with no GPU becomes the Master or just a browser and still trains on lab GPUs.

11. Advantages

Free and local

Uses hardware the college already owns. No cloud bills, no Colab session limits, data stays inside the private network.

Zero-install and portable

No admin rights, Docker or WSL. Each PC is ready ~3 min after plugging in the stick; removal is deleting a folder.

Works across networks

Tailscale connects PCs in different rooms, buildings, or on hotspots, without router changes.

Familiar interface

Colab-like notebook, automatic library installs, Jupyter-compatible `.ipynb` files.

Shared by many users

Everyone uses the same dashboard at once; Ray hands out GPU shares fairly and refuses clearly when full.

Fault tolerant where it counts

Resumable, verified transfers; atomic installs; a stop script that never touches the user's own Python; offline fallback to cached data.

Observable

Detailed per-GPU telemetry and history make it obvious which PC is busy, hot or throttling.

Consistent environments

Every node runs a byte-identical environment, removing "works on my machine" problems.

12. Drawbacks and limitations

Being honest about these is important when presenting the project.

Limitation	Explanation / mitigation
One job = one GPU	It runs many jobs in parallel but does not split *one* model across PCs. That needs distributed-training code (`torch.distributed`), and consumer networks are far too slow for gradient exchange between PCs.
Single Master	The Master is a single point of failure: stopping it ends every running notebook and job. No automatic fail-over.
Ray on Windows is beta	Multi-node Windows clusters are officially unsupported by Ray (enabled with `RAY_ENABLE_WINDOWS_OR_OSX_CLUSTER=1`). It worked in testing, but edge cases are possible.
No authentication	Anyone who can reach port 8000 on the tailnet can run any code on every worker and read every dataset. Only invite trusted people; use Tailscale ACLs.
Network-bound data	If Tailscale relays traffic (DERP), transfers drop to a few MB/s. Mitigation: USB-seeded datasets, prefetching before class.
Sleep and shutdown	A sleeping or rebooted PC drops out; its running work fails. Lab PCs must stay on with sleep disabled.
NVIDIA only	The CUDA build needs NVIDIA GPUs and drivers ≥ 527.41 (a CUDA 11.8 build covers older drivers). AMD/Intel GPUs can only contribute CPU.
Disk and RAM use	~5 GB per PC for the environment, plus dataset caches; each Ray node uses ~0.5–1 GB RAM (tight on 8 GB PCs).
Windows quirks	`DataLoader(num_workers>0)` can't re-import code sent as a string, so use `num_workers=0` ; per-process VRAM isn't reported under WDDM.
Auto-install needs internet	Offline PCs can't install missing libraries; add them to the build and redistribute the stick.
Notebooks live in the browser	Stored in localStorage; export important ones as `.ipynb` .
Scale tested	Tested end-to-end on one and two PCs; a full lab of many PCs has not been load-tested.

13. Comparison with alternatives

Alternative	What it offers	Why TensorSwarm instead (or not)
Google Colab / Kaggle	Free cloud GPUs, zero setup, polished notebooks.	Session and usage limits, shared/queued GPUs, data must be uploaded to Google. TensorSwarm uses local hardware with no quotas, but you manage the PCs yourself.
University HPC (SLURM)	Powerful, managed, multi-GPU nodes, fast interconnects.	Needs accounts, queues, Linux skills and dedicated servers. TensorSwarm needs none, but is far less powerful and not centrally managed.
Docker / Kubernetes cluster	Industry-standard orchestration, isolation, scaling.	Needs admin rights, Linux or WSL, and a lot of setup; not possible on locked-down Windows lab PCs. (TensorSwarm still ships Linux/Docker scripts for Linux nodes.)
Plain Ray cluster	The same scheduler.	TensorSwarm *is* Ray plus the parts students need: packaging, networking, install, telemetry, notebook, datasets, UI.
Horovod / torch.distributed	Train one big model across many GPUs.	Different goal (data-parallel training of one model) and needs fast networks. Complementary, not a replacement.
BOINC / volunteer computing	Huge numbers of donated PCs for embarrassingly parallel science.	Batch-only, no interactive notebook, high latency. TensorSwarm is interactive and private.

> **Positioning in one line**
>
> TensorSwarm sits between Colab (easy but limited) and an HPC cluster (powerful but heavy): an interactive, private GPU service built from hardware a college already has, deployable by students in an afternoon.

14. Likely questions, with answers

Q: Why "peer-to-peer" if there is a Master?
The network is a peer-to-peer mesh: Tailscale connects every PC directly to every other, with no central VPN server carrying traffic. The Master coordinates scheduling, but any PC can be the Master, and any PC on the mesh can use the cluster.

Q: Can it train one model on all the GPUs together?
No, by design. It runs many independent jobs in parallel. Data-parallel training of one model needs gradients exchanged every step; over Wi-Fi or a VPN that communication would cost more than the extra GPUs save.

Q: What happens if a worker PC is switched off mid-job?
Its heartbeats stop, the dashboard marks it OFFLINE after 15 s, and Ray reports the task or actor as failed. Other workers are unaffected. The user restarts the runtime, which lands on another free GPU.

Q: How does the scheduler choose a GPU?
Ray keeps a count of free GPU shares per node. A request for `num_gpus=1` goes to a node with a whole free GPU. A specific PC can be pinned with node affinity, and jobs that need a dataset prefer a PC that already holds it (soft affinity).

Q: Why not just use Docker?
Lab PCs don't allow it: Docker Desktop needs admin rights, WSL2 or Hyper-V, and a heavy install. The portable environment runs as a normal user from a folder.

Q: Is it secure?
Traffic is encrypted by WireGuard and the firewall only admits Tailscale addresses. But there is no login: anyone on the tailnet can run code on the workers. That is acceptable for a trusted class group; it would need authentication before wider use.

Q: How do you guarantee all PCs run the same software?
They are all unpacked from the same archive, identified by a hash of `pip freeze`. Ray itself refuses to connect nodes with different versions, so a mismatch is detected immediately.

Q: How does a 10 GB dataset reach 20 PCs over slow Wi-Fi?
It shouldn't go over Wi-Fi. Big datasets go on the USB stick and are seeded onto every PC during install. Medium ones are uploaded once and prefetched in parallel before class. Every transfer resumes and is checksum-verified, so interruptions only cost time.

Q: What was the hardest problem?
Two stand out: (1) CUDA calls from different Ray threads froze the process on Windows, fixed with one dedicated cell thread; (2) workers crashed because Ray reused the Master's user-profile temp path, fixed with a shared `C:\ProgramData` path.

Q: What would you add next?
Authentication and per-user quotas; Master fail-over; peer-to-peer dataset sharing between workers; automatic cache eviction; saving notebooks on the server; support for distributed training on wired lab networks.

Q: How is this different from Colab?
Colab gives you someone else's GPU with limits. TensorSwarm gives you your college's GPUs with no session limits and a similar notebook, at the cost of managing the PCs and Tailscale yourselves.

15. Glossary

Term	Meaning
Master / head node	The PC running the Ray head and the FastAPI server; coordinates everything.
Worker node	A PC that offers its GPU to the cluster and runs user code.
Tailscale / tailnet	A mesh VPN built on WireGuard; the tailnet is the private network of your PCs.
WireGuard	A modern, fast, encrypted VPN protocol.
DERP relay	Tailscale's fallback relay server when two PCs can't connect directly; slower.
NAT traversal	Techniques that let two PCs behind routers connect to each other directly.
Ray task	A function executed remotely once (used for batch jobs).
Ray actor	A long-lived remote process with state (used for notebook runtimes).
Node affinity (hard / soft)	Asking Ray to run on a specific node: hard = must, soft = prefer.
Fractional GPU	Reserving part of a GPU (e.g. 0.5) so two workloads share it.
Heartbeat	A periodic "I'm alive" message carrying metrics, sent every 5 s by each agent.
NVML	NVIDIA Management Library: reads GPU metrics.
CUDA	NVIDIA's GPU programming platform used by PyTorch.
VRAM	The GPU's own memory; limits model and batch size.
TFLOPS	Trillions of floating-point operations per second; raw compute speed.
Content addressing	Naming data by a hash of its content, so identical data has the same name.
SHA-256	A cryptographic hash used to verify that files arrived intact.
HTTP Range request	Asking a server for only part of a file, used to resume downloads.
Atomic operation	A change that happens entirely or not at all (e.g. a folder rename).
Portable Python	A Python that runs from any folder without installation.
Static export	Compiling a web app to plain HTML/JS files any web server can serve.
WDDM	The Windows display driver model; it hides per-process VRAM usage.